

ACTIVIDAD 4

DESAROLLO DE
APLICACIONES
MOVILES

ABISH ABRIL
SANTANA
GARCÍA

JUAN ESTEBAN
CAMPOS CRUZ

ROJAS PEREA,
ADALBERTO
EMMANUEL



Descripción

Los equipos crearán una aplicación de gestión de notas que incluya:

Animaciones: transiciones al agregar/eliminar notas, feedback visual al completar tareas.

Gestos: deslizar para eliminar y zoom con pellizco en imágenes adjuntas.

Base de datos local: persistencia de notas usando Room, con operaciones CRUD.

Interfaz adaptable: diseño responsivo para pantallas pequeñas y grandes.

Objetivo

Desarrollar una aplicación móvil que combine animaciones fluidas, gestos intuitivos y almacenamiento local con Room, priorizando una experiencia de usuario cohesiva y eficiente.

1. Configuración inicial

```
@Entity(tableName = "tareass")
data class Event(
    @PrimaryKey(autoGenerate = true)
    val id: Long = 0,
    val title: String,
    val date: String,
    val time: String = "",
    val priority: Priority,
    val description: String,
    val isCompleted: Boolean = false,
    val createdAt: Long = System.currentTimeMillis()
)
```

En esta parte se crea la entidad con Room para definir la tabla de tareas dentro de la base de datos. Aquí se configura los campos que va a almacenar cada registro: su identificador único autogenerado, título, fecha, prioridad y si la tarea ya fue completada.

2. Implementar operaciones CRUD

```
interface TareaDao {
    1 Implementation
    @Query(value = "SELECT * FROM tareas ORDER BY createdAt DESC")
    fun getAll(): Flow<List<Event>>

    1 Implementation
    @Insert
    suspend fun insert(event: Event)

    1 Implementation
    @Update
    suspend fun update(event: Event)

    1 Implementation
    @Delete
    suspend fun delete(event: Event)
```

Aquí definimos la interfaz para conectar la aplicación con la base de datos de Room. Se agregan las funciones básicas del CRUD que me permiten consultar todas las tareas guardadas, así como insertar, actualizar y borrar elementos sin trabar la pantalla.

3. Diseñar interfaz con animaciones

```
7 LaunchedEffect(key1 = Unit) { visibleState.targetState = true }
8
9
10 val dismissState = rememberSwipeToDismissBoxState(
11     confirmValueChange = { targetValue ->
12         val isDismissed =
13             targetValue == SwipeToDismissBoxValue.EndToStart ||
14             targetValue == SwipeToDismissBoxValue.StartToEnd
15         if (isDismissed) {
16             onDelete(event)
17         }
18         isDismissed
19     }
20 )
21
22 val cardColor by animateColorAsState(
23     targetValue = if (event.isCompleted) {
24         MaterialTheme.colorScheme.surfaceVariant
25     } else {
26         MaterialTheme.colorScheme.surface
27     },
28     animationSpec = tween(durationMillis = 400),
29     label = "cardColor"
30 )
31
32 AnimatedVisibility(
33     visibleState = visibleState,
34     enter = fadeIn(animationSpec = tween(durationMillis = 300)) + slideInVertically(
35         animationSpec = tween(durationMillis = 200)),
36     exit = fadeOut(animationSpec = tween(durationMillis = 200)),
37     modifier = modifier
38 ) {
```

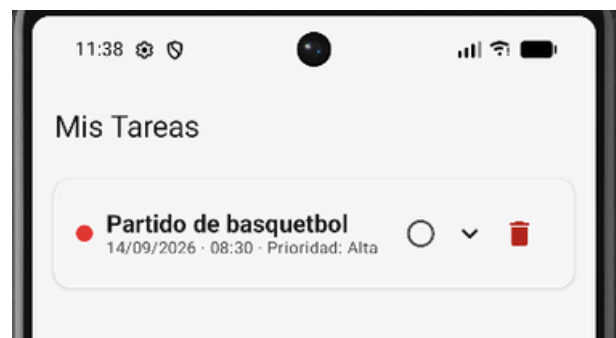
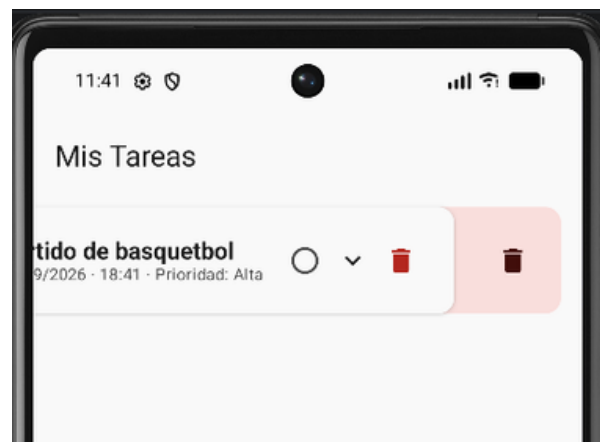
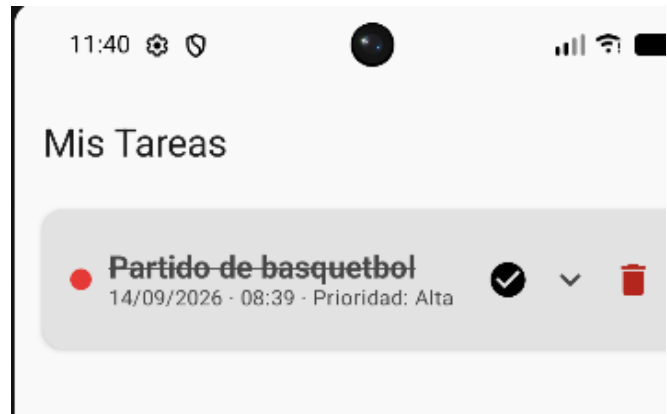
Aquí se implementa la parte visual interactiva con Jetpack Compose. Se agrega el gesto de deslizar para eliminar elementos `SwipeToDismissBox`, una animación suave que cambia el color de la tarjeta según si la tarea está completada o no, y transiciones de entrada y salida `AnimatedVisibility`.

4. Añadir gestos

```
SwipeToDismissBox(
    state = dismissState,
    backgroundColor = {
        Box(
            modifier = Modifier
                .fillMaxSize()
                .padding(vertical = 6.dp)
                .clip(shape = RoundedCornerShape(size = 12.dp))
                .background(color = MaterialTheme.colorScheme.errorContainer)
            contentAlignment = Alignment.CenterEnd
        ) {
            Icon(
                imageVector = Icons.Default.Delete,
                contentDescription = null,
                modifier = Modifier.padding(end = 24.dp),
                tint = MaterialTheme.colorScheme.onErrorContainer
            )
        }
    }
) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp, vertical = 8.dp)
```

En esta sección se configurará el gesto de tamaño reducido `SwipeToDismissBox` para que el usuario pueda borrar una tarea al arrastrarla. Se agregará un fondo con color de alerta y un icono de papelera alineado al extremo, dando una respuesta visual clara de eliminación al deslizar la tarjeta.

5. Pruebas



Diseño en Figma

<https://www.figma.com/design/IE1UfjrCgl8DxXm1qpySDi/Actividad-3-4?node-id=134-3647&t=Bl8NvpfDNH40989A-1>

<https://www.figma.com/proto/IE1UfjrCgl8DxXm1qpySDi/Actividad-3-4?node-id=459-728&t=9hfTHMzFtMpTgluo-1&scaling=scale-down&content-scaling=fixed&page-id=134%3A3647&starting-point-node-id=459%3A797>

Actividad en clase

Configuración inicial

Crea un nuevo proyecto en Android Studio con Jetpack Compose.

Agrega las dependencias de Room al archivo build.gradle (usar versión 2.7.1 o superior).

```
plugins {  
    id("com.android.application")  
    id("org.jetbrains.kotlin.android")  
    // Este plugin habilita el "compilador de Compose" para Kotlin 2.0+.  
    id("org.jetbrains.kotlin.plugin.compose")  
    // Genera en tiempo de compilación el código del DAO/Database de Room.  
    id("com.google.devtools.ksp")  
}
```

```
dependencies {  
    // --- Núcleo de Android + integración de Compose con Activity/ViewModel ---  
    implementation("androidx.core:core-ktx:1.13.1")  
    implementation("androidx.activity:activity-compose:1.9.2")  
    implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.8.6")  
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.6")  
}
```

Modelado de datos:

Define una entidad Task con campos: id (autogenerado), title (String), description (String), isCompleted (Boolean) y dueDate (Long).

Crea un DAO con métodos para insertar, eliminar, actualizar y listar tareas.

(usar Flow para reactividad).

```
@Entity(tableName = "tareass")  
data class Event(  
    @PrimaryKey(autoGenerate = true)  
    val id: Long = 0,  
    val title: String,  
    val date: String,  
    val time: String = "",  
    val priority: Priority,  
    val description: String,  
    val isCompleted: Boolean = false,  
    val createdAt: Long = System.currentTimeMillis()  
)
```

```
interface TareaDao {  
    1 Implementation  
    @Query("SELECT * FROM tareass ORDER BY createdAt DESC")  
    fun getAll(): Flow<List<Event>>  
  
    1 Implementation  
    @Insert  
    suspend fun insert(event: Event)  
  
    1 Implementation  
    @Update  
    suspend fun update(event: Event)  
  
    1 Implementation  
    @Delete  
    suspend fun delete(event: Event)
```

Actividad en clase

Configuración de la base de datos

Implementa una clase AppDatabase abstracta con patrón Singleton.

Define un repositorio (TaskRepository) que use el DAO para operaciones.

```
@Database(entities = [Event::class], version = 2, exportSchema = false)
@TypeConverters(...value = Converters::class)
abstract class AppDatabase : RoomDatabase() {

    2 Usages 1 Implementation
    abstract fun tareaDao(): TareaDao

    4 Usages
    companion object {
        2 Usages
        @Volatile
        private var INSTANCE: AppDatabase? = null

        1 Usage
        fun getDatabase(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(lock = this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    class = AppDatabase::class.java,
                    name = "tareass_database"
                )
                    .fallbackToDestructiveMigration()
                    .build()
                INSTANCE = instance
            }
        }
    }
}
```

```
interface TareaDao {
    7 Usages 1 Implementation
    @Query(value = "SELECT * FROM tareass ORDER BY createdAt DESC")
    fun getAll(): Flow<List<Event>>

    5 Usages 1 Implementation
    @Insert
    suspend fun insert(event: Event)

    2 Usages 1 Implementation
    @Update
    suspend fun update(event: Event)

    2 Usages 1 Implementation
    @Delete
    suspend fun delete(event: Event)
}
```

Integración con ViewModel

Crea un TaskViewModel que exponga los datos mediante StateFlow y maneje eventos como addTask o deleteTask. Usa viewModelScope para operaciones asíncronas.

```
val events: StateFlow<List<Event>> = dao.getAll().stateIn(
    scope = viewModelScope,
    started = SharingStarted.WhileSubscribed(stopTimeoutMillis = 5_000),
    initialValue = emptyList()
)
```

```
fun addTarea(title: String, date: String, time: String, priority:
    viewModelScope.launch {
        dao.insert(
            Event(
                title = title,
                date = date,
                time = time,
                priority = priority,
                description = description
            )
        )
    }
}
```

Actividad en clase

Diseño de UI

Implementa una pantalla con:

LazyColumn para mostrar tareas.

FloatingActionButton para abrir un diálogo de creación.

Switch para marcar tareas como completadas.

← Nuevo Tarea

Título

Salida con Michelle

Fecha

24/09/2026

Hora

21:30

Prioridad

Media

Descripción

Ir a correr con Michelle

Guardar tarea

Mis Tareas

~~Salida con Michelle~~

24/09/2026 · 21:30 · Prioridad: Media

✓ ▼ 🗑

Mis Tareas

Ir con Fernanda

19/09/2026 · 09:00 · Prioridad: Alta

○ ▼ 🗑

Salida con Michelle

24/09/2026 · 21:30 · Prioridad: Media

○ ▼ 🗑

Uso de IA

El uso de IA se implemento para investigar y ver como se implementa un nuevo diseño de agregar tareas y la opción de usar el calendario y hora de la librería.